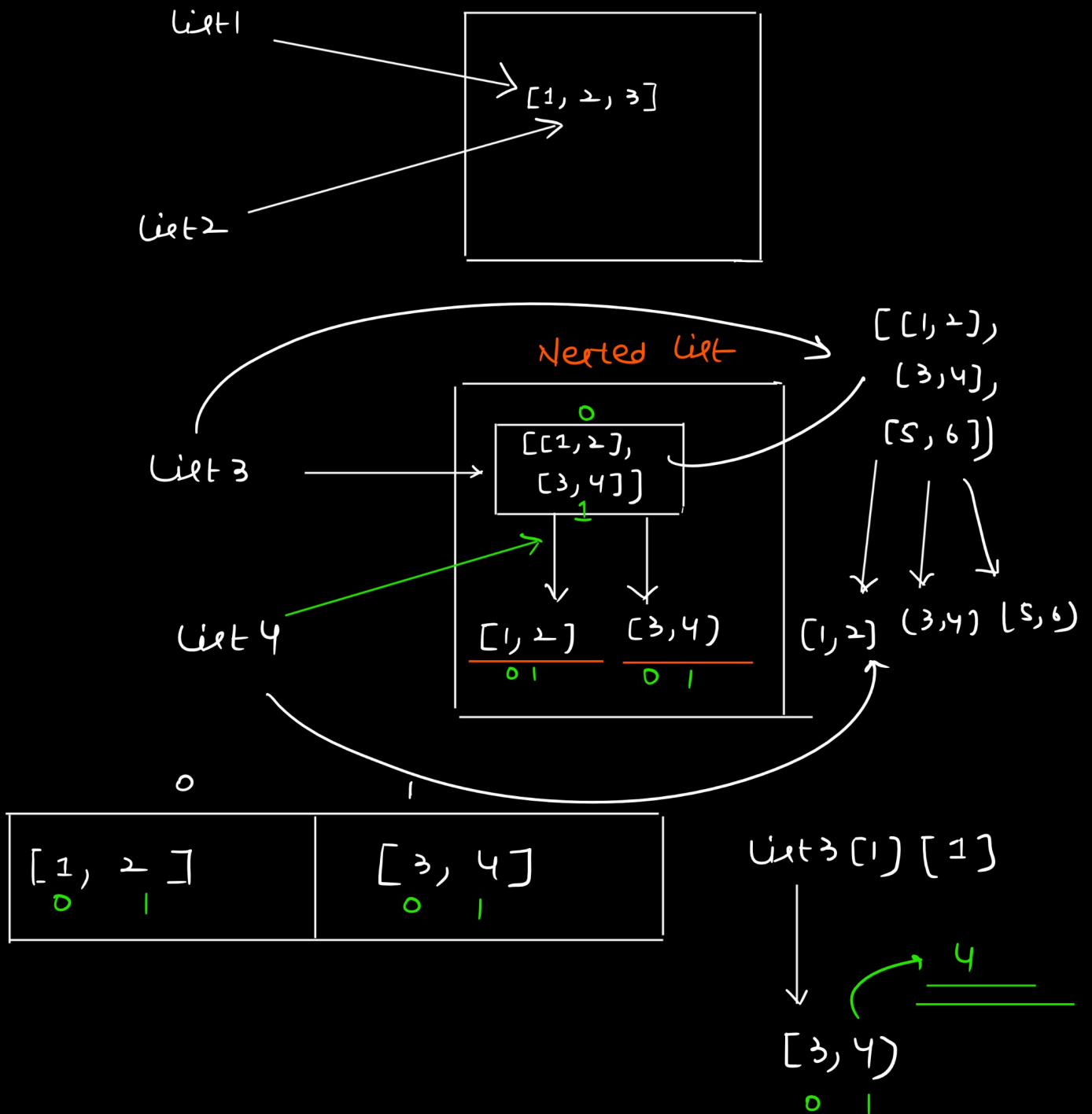
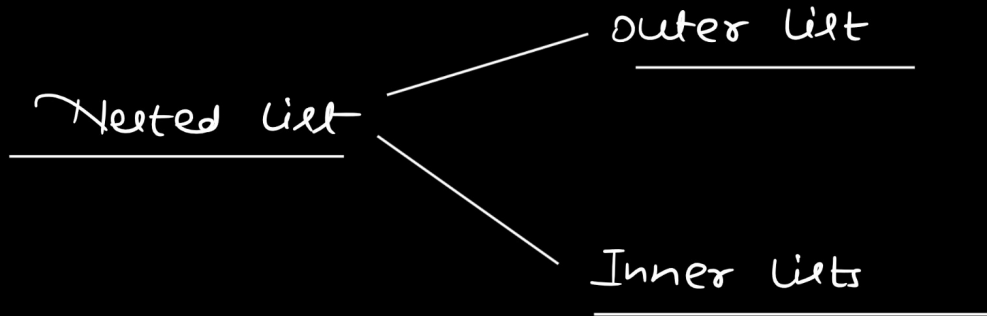
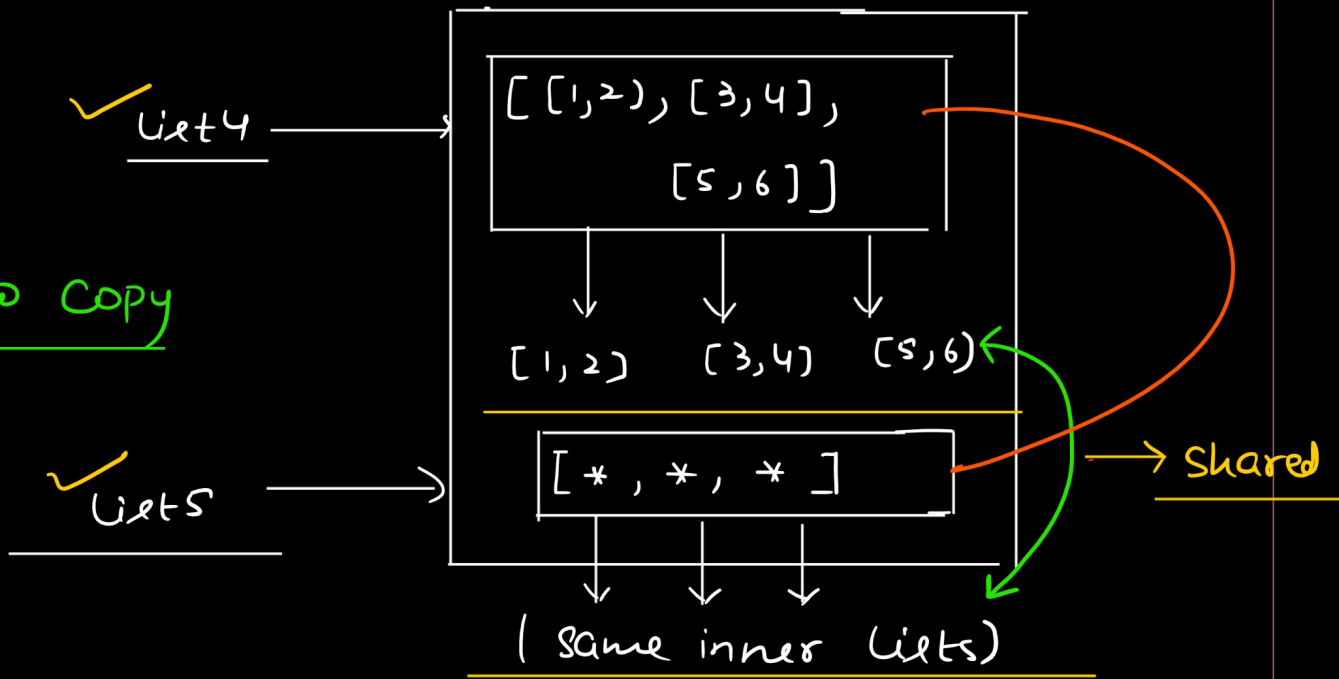


Python internal working

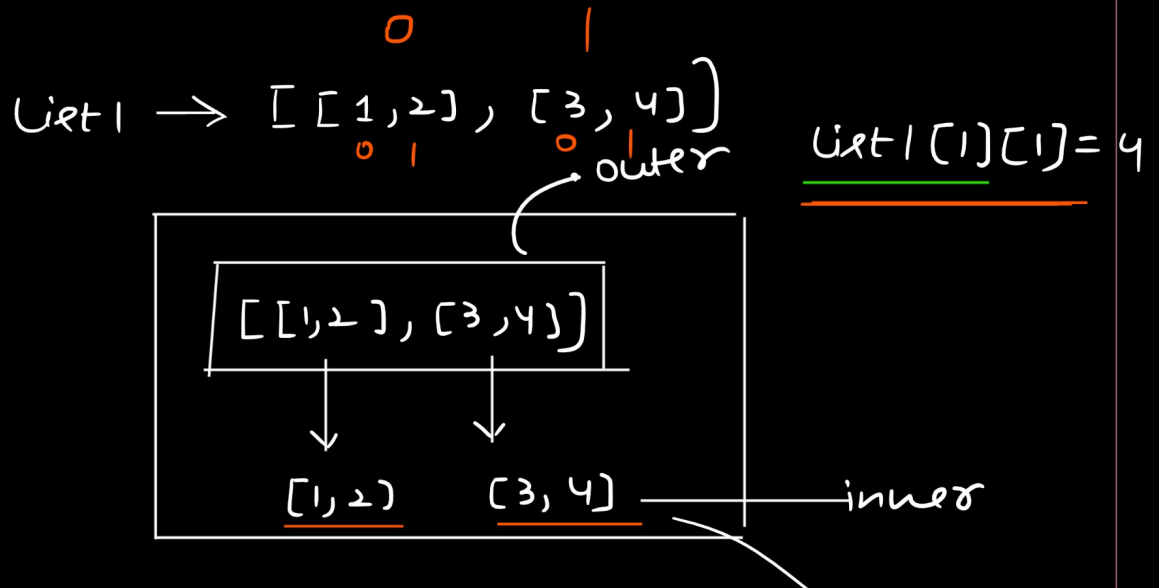
List → Mutable data structure

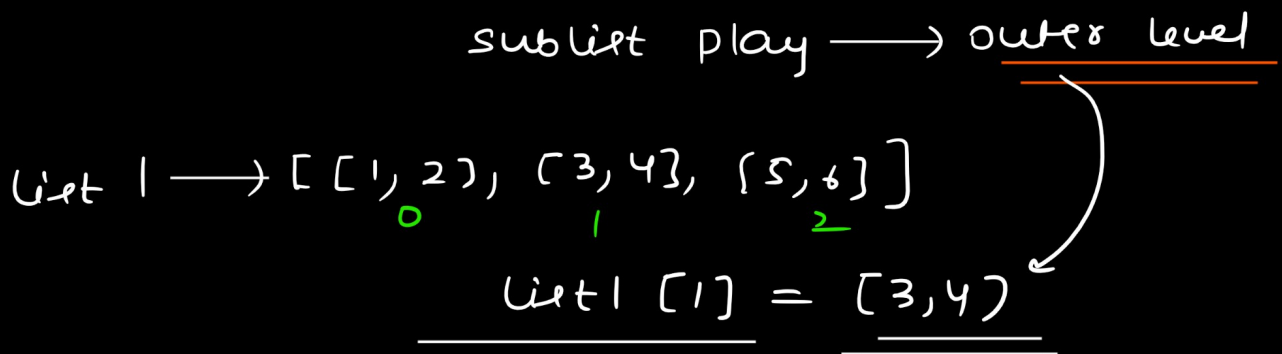
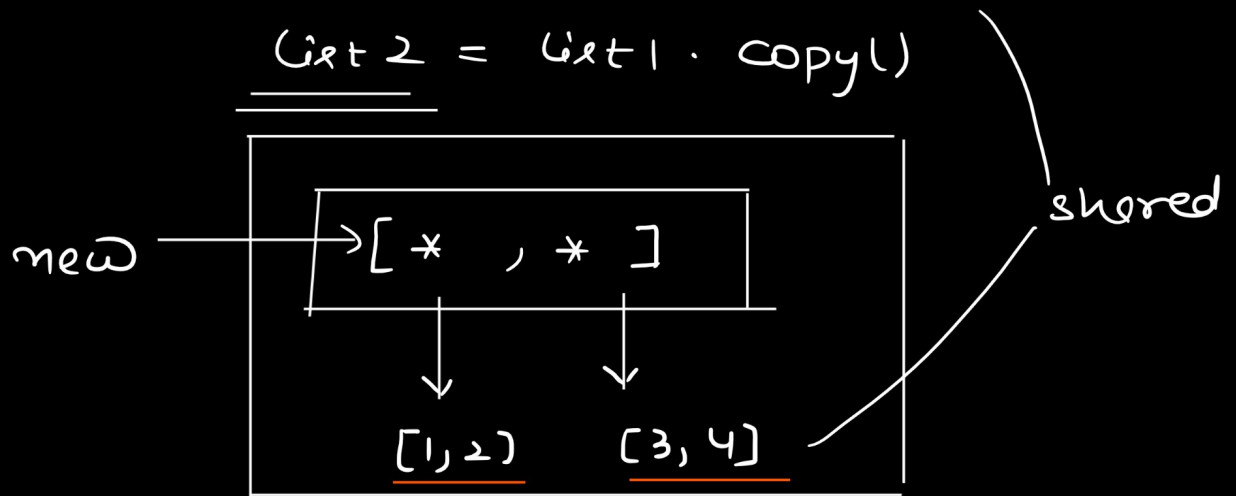


shallow copy

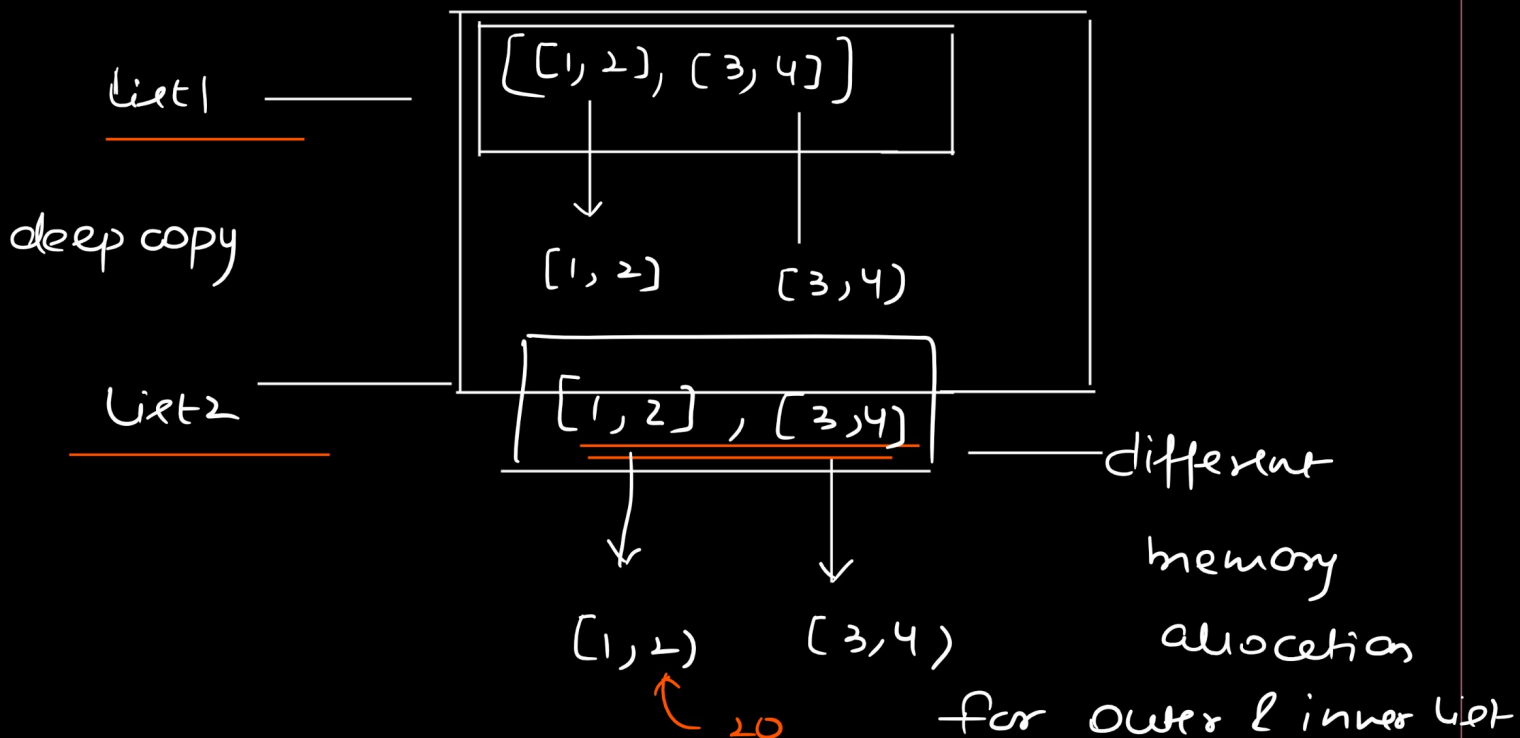


Shallow copy → Outer list is new,
inner list shared





List 1 [1] [1] = 4 (Internal level)



Tuple → Immutable

↳ faster ds as comparable
to the list

Dynamic Data / Static Data
↳ list

tuple

{ retrieval of elements
or
analyze the data

Quick Note:

Lists vs Tuples

1

Lists are mutable, meaning their elements can be modified after creation. You can add, remove, or change elements in a list.

Tuples, on the other hand, are immutable, meaning once they are created, their elements cannot be changed, added, or removed.

2

Lists are created using square brackets [].

Tuples are created using parentheses ().

3

Lists are slightly slower than tuples due to their mutability. When elements are added, removed, or modified in a list, it may require more memory operations.

Tuples are generally faster than lists because they are immutable. Once created, the elements in a tuple remain constant, resulting in better performance.

4

```
my_list = [1, 2, 3]
```

```
my_tuple = (1, 2, 3)
```

5

Due to their mutable nature, lists may consume more memory over time, especially as elements are added or removed.

Since tuples are immutable, they have a fixed size and occupy a constant amount of memory.

Sets → unordered,
mutable, unindexed,
 { }
 Dictionary → d { key: value }
 does not allow duplicate elements.
 ↳ unique item

Quick Note :-

Aspect	Lists	Tuples	Sets
Mutability	Changeable ✓	unchangeable ✓	Changeable ✓
Indexing	Yes ✓	Yes ✓	No ✓
Duplicate	Allowed ✓	Allowed ✓	Not Allowed ✓
Order	Ordered ✓	Ordered ✓	Unordered ✓
Syntax	Enclosed in square brackets [] ✓	Enclosed in parentheses () ✓	Enclosed in curly braces { } ✓

Immutable

Mutable

Dictionary

Enrollment Id , Roll No

key (unique) → Value

EmpID →
 (key) all
 the

details of
 employee

{
 firstName, lastName,
 fatherName, motherName,
 Address, emailId,
 PhoneNum